

I used logistic regression to classify documents as baseball or hockey. The input features were word counts plus a bias term; I did not use TF-IDF. The model predicts the probability that a document is about baseball using a sigmoid function, then adjusts the weights with stochastic gradient updates. All runs used the same 1,064 training documents and 133 held-out documents, with random seed 1701.

I compared learning rates of 0.01, 0.1, and 0.5 over ten passes through the training data. After the first pass, their held-out accuracies were 91.73%, 93.98%, and 93.23%, respectively. By pass ten, these were 94.74%, 94.74%, and 93.23%. The rate of 0.01 took longer to learn, while 0.5 fluctuated more early on. Of the three, 0.1 gave the lowest final held-out mean log loss: 0.145, compared with 0.185 for 0.01 and 0.238 for 0.5. So even though 0.01 and 0.1 ended with the same accuracy, 0.1 made better probability predictions by this measure.

At a learning rate of 0.1, training accuracy reached about 99.91% by pass four and stayed there. Held-out accuracy was around 94.74% from pass two onward, with a brief increase to 95.49% at pass five. Four passes were enough for accuracy to settle down in this run. That does not mean the weights or loss had stopped changing, though.

For the extra credit, I tried lazy L2 regularization and a decreasing learning rate. With lazy regularization, I used a learning rate of 0.1 and a regularization parameter of 0.001. Regularization was delayed for inactive features, and any remaining decay was applied before evaluation. This gave 96.24% held-out accuracy during passes three through seven, but it dropped to 93.98% by pass ten, with a final loss of 0.173. The early gain did not last. The decreasing rate was 0.1 divided by the square root of the update number. It learned more slowly, reaching 95.58% training accuracy and 94.74% held-out accuracy after ten passes. Its held-out loss was 0.297, so matching the constant-0.1 model's accuracy did not mean it matched the quality of its probability predictions.

The trained weights show which words influence the predictions. For the constant-0.1 model after ten passes, ranking coefficients with the bias excluded gives "runs" (+1.815), "pitching" (+1.523), and "baseball" (+1.217) as the strongest baseball predictors. The most negative coefficients, "hockey" (-2.925), "playoffs" (-1.996), and "golchowy" (-1.547), favor hockey. Mathematically, $\log(p / (1 - p)) = b + \sum(w_j x_j)$, so each additional occurrence of a word changes the baseball log-odds by its weight, holding other counts fixed. Ranking words seen in training by increasing absolute weight identifies weak predictors: "dive," "privately," and "kills" each had weights approximately zero. They contribute almost nothing to either class in this fitted model.

There are a few limits to these results. Word counts leave out word order and context, and longer documents can have a large effect on the updates. I also reused one small held-out split for all comparisons. A single document changes its accuracy by about 0.75 percentage points, so small differences should be treated cautiously. Training accuracy was almost perfect, but held-out accuracy remained lower. The lazy implementation also still scans every feature, and applying the delayed decay frequently limits the time it could save. These experiments did not establish a speed advantage for it.